

R2 Mock — LeetCode 2025 Report

30 minutes · Technical Interview 2 · Summer Intern 2026

Mapped to your resume. Verdict in report: Selected, Mumbai, Rs. 50k/month.

From report: LeetCode Discuss, Sep 2025: R2 = brief intro, casual tone, deep resume/projects, design choices + challenges, behavioral (team, conflicts, new stack), balanced technical + cultural. No hard DSA in R2. HR later asked layman project explanation.

Interviewer tone: starts casual, puts you at ease, then goes deep on whatever is on the resume.

How to use. 30-minute timer. Answer out loud first; green boxes are the target answers mapped to your projects.

R2 intro: Same 40-second version as Dilip mock (LogiFlow pipelines + Community Hero + Nexus sixteen).

This mock covers R2 only. R1 topics (SSR, SQL join, max-in-string) appear in the bonus appendix if R1 and R2 blur on your drive day.

1. Clock (matches report structure)

0:00–3:00	Intro + rapport	Brief intro; casual chat to settle in.
3:00–12:00	Resume / LogiFlow	Tech stack, design choices, challenges.
12:00–18:00	Community Hero	Solo project, AI, auth, trade-offs.
18:00–25:00	Behavioral	Team conflict, new stack, collaboration.
25:00–30:00	Wrap-up	Your questions; thank you.

2. Script — Technical Interview 2

0:00 — Opening

Interviewer. *Hi Ojas, how are you doing? Hope round one went okay. I will keep this relaxed — just want to understand your projects and how you work in a team. Quick intro?*

You say. R2 intro. Stop.

Interviewer. *Nice. Let us start with LogiFlow. Walk me through the tech stack and why you picked each piece.*

You say. Backend Python + FastAPI — data pipelines and sklearn delay model. Frontend TypeScript + Next.js — map UI with faster first load. SQL as source of truth for corridors and tariffs. Redis cache-aside for hot reads, 100–400 ms. pytest 100/100 business rules in CI. Deployed on GCP Cloud Run. Each choice matched the constraint: data + ranking on the backend, read-heavy API, tests as contracts.

Interviewer. *What was the hardest challenge while building it?*

You say. Latency. First version recomputed rankings on every request — seconds. I moved to cache-aside with Redis and kept SQL as source of truth. That took responses to 100–400 milliseconds. Second challenge was team alignment on what “co-lead” owns — we fixed it by naming pipeline owners in the README.

Interviewer. *LogiFlow was a team project. Any conflict with a teammate? How did you handle it?*

You say. Cache placement — API layer vs pipeline layer. We built both on a branch, benchmarked, picked API-level for simpler invalidation with the same latency. No personal conflict; we let data decide.

Interviewer. *Design choice question: why SQL database, not a document store like MongoDB?*

You say. Corridors, legs, and tariffs are relational — joins and foreign keys. One tariff update is one row, not thousands of denormalized documents. Mongo fits flexible blobs; my Community Hero reports are exactly that, so that project uses Firestore instead.

12:00 — Second project (they often switch)

Interviewer. *Community Hero — you said you built it solo. Tell me about that. Tech stack and why.*

You say. Civic issue reporting PWA. React + Express, Firestore for geo-tagged reports, Cloud Run deploy, Gemini for category classification. Solo because hackathon timeline — I owned spec to deploy. Firestore because each report is a self-contained document with nested fields; schema was still moving. Gemini behind guard checks — bad output falls back to manual admin review.

Interviewer. *What if Gemini is wrong or the API is down?*

You say. Treat the model as untrusted. Validate input before the call. If output is not a known category, store raw text and flag for admin. Prefer a missed auto-tag over a wrong auto-route. If Gemini is down, reports still save; routing goes to manual queue.

Interviewer. *How did you handle login and authentication?*

You say. Firebase Auth on the client issues a token. Express verifies it on every protected route before Firestore. Roles checked server-side — a citizen cannot hit admin endpoints. I never trust the client for authorisation.

18:00 — Behavioral (quoted in report)

Interviewer. *Would you prefer to keep working in Python and React, or are you open to a completely new tech stack in a corporate setting?*

You say. Open to a new stack. GEP often uses C#, Angular, SQL Server — I have not shipped those, but I learned FastAPI, Redis, and Cloud Run under deadline for LogiFlow. The habits transfer: APIs, data, tests, ownership. I would rather learn the product stack than insist on mine.

Interviewer. *How do you collaborate on tasks when the team is under deadline?*

You say. Name owners early — on LogiFlow we split pipelines by person and shared a ranker interface. Daily sync on blockers, PR reviews before merge. On Nexus, sixteen-person core team — divide events, one owner per deliverable. I do not merge what I have not reviewed.

Interviewer. *If I asked you to explain one project to someone with zero technical background, which would you pick and how?*

You say. LogiFlow. “You need to ship goods from city A to B. Train, truck, or both — different price, time, and delay risk. LogiFlow shows all options ranked so you pick what fits your budget and deadline — like Google Maps for freight, but you see trade-offs, not just one fastest route.”

*Report says this layman question appeared in **HR**; it may come in R2 if time allows. Have it ready either way.*

25:00 — Wrap-up

Interviewer. *That is most of what I had. Anything you want to ask me?*

You say. For technology interns, what does the first month look like — shadowing, or contributing to a live module? And what separates interns who get a PPO from those who do not?

Interviewer. *Good questions. Thank you, Ojas — HR will connect with you on next steps.*

You say. Thank you. I enjoyed this.

3. Bonus — if R1 topics repeat (from same LeetCode report)

R1 was 30 min in the report. If your drive merges rounds or Dilip asks R1-style questions, drill these once:

Interviewer. *What is SSR in Next.js?*

You say. Server renders React to HTML before sending to the browser, so the user sees content immediately. Plain React sends empty HTML + JS first, then fetches data — blank screen longer. LogiFlow map benefits from data in the first paint.

Interviewer. *Stack vs Queue?*

You say. Stack LIFO — undo, DFS, call stack. Queue FIFO — BFS, job scheduling. LogiFlow graph search uses a queue for BFS on corridors.

Interviewer. *SQL: books and authors — find books by author X.*

You say. `SELECT b.name FROM book b JOIN author a ON b.author_id = a.author_id WHERE a.name = 'X';`

Interviewer. *String $s = "1,2,3,4,5"$ — find maximum integer.*

You say. Split on comma, parse each token to int, track max in one pass. $O(n)$ time, $O(1)$ extra if split in place. Watch empty string and non-numeric tokens — ask if input is guaranteed clean.

4. Score this mock

Rapport	Short intro, not a monologue? Comfortable tone?	_____
LogiFlow	Stack + why each? Challenge + fix? SQL vs NoSQL?	_____
Community	Solo ownership clear? Gemini guards + fallback?	_____
Hero		
Behavioral	New stack = yes + example? Conflict = evidence?	_____
Layman	No jargon in dinner-table explanation?	_____
Close	Two thoughtful questions, not stipend?	_____

Report reminder. Selected candidate: R2 had **no** heavy coding. Real filter was project depth + team fit. HR asked family + layman project — prep those even if R2 feels “easy.”

Pair with: GEP_R2_Mock_Dilip.pdf and GEP_R2_LogiFlow_Prep.pdf.